claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf_d88615a4-f56\agent-a98897151b5da076f.jsonl`
- SHA-256: `70295e2fa165dc0e0748dfb9f8ce366f1980b248f41adace2cbb2e23f9b97489`
- Source modified: `2026-06-07T09:12:01+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_d88615a4-f56`
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`

Transcript

1. user (2026-06-07T09:10:30.780Z)

Read these files in C:/Users/avidu/Projects/badminton-highlight-indexer and produce a DENSE, anchor-rich CODE_MAP chunk for the "Orchestration & config" subsystem.
Use the format: bullet per file/symbol with anchors written as @path::symbol where @path is the repo-relative path EXACTLY as listed (the current layered layout). Capture: key classes/functions/consts; data contracts (mark with §); registries / plugin extension points (mark with ?); and footguns/gotchas (mark with ?). Be precise ? only list symbols that actually exist in the files. If a path is a directory (e.g. tests/), enumerate its files and map each test to the subsystem it covers.
Files/paths: backend/main.py, backend/config/__init__.py, backend/config/models.py, backend/config/loader.py, run_eval.py, run_phase3_eval.py, run_regression.py, run_process_and_stitch.py, setup.py, config.json

2. assistant (2026-06-07T09:10:33.809Z)

I'll read all the specified files in parallel to produce the CODE_MAP chunk.

3. user (2026-06-07T09:10:34.330Z)

```
1       import os
2       import sys
3       import argparse
4       import hashlib
5       import json
6       import uvicorn
7       from dotenv import load_dotenv
8
9       load_dotenv()
10      from fastapi import FastAPI, HTTPException
11      from fastapi.middleware.cors import CORSMiddleware
12      from pydantic import BaseModel
13      from typing import List, Dict, Any, Optional
14
15      from fastapi.staticfiles import StaticFiles
16      from fastapi.responses import HTMLResponse
17      from backend.pipeline.validators.base import registry
18      from backend.infrastructure.database import Database
19      from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
20      from backend.pipeline.stitcher.compiler import VideoCompiler
21      from backend.config import load_config as load_app_config, AppConfig  # new typed config
22      from backend.reporting import emit_indexer_report
23
24      app = FastAPI(title="Sports Highlight Indexer API")
25
26      # Enable CORS for local web interface
```

```
    app.add_middleware(
        CORSMiddleware,
        allow_origins=["*"],
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )

    # Serves static frontend files directly (now under api/ per approved structure)
    os.makedirs("backend/api/static", exist_ok=True)
    app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")

    @app.get("/", response_class=HTMLResponse)
    def serve_index():
        index_path = "backend/api/static/index.html"
        if os.path.exists(index_path):
            with open(index_path, "r", encoding="utf-8") as f:
                return f.read()
        return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"

    # Global variables initialized at startup
    db_instance: Optional[Database] = None
    global_config: Optional[AppConfig] = None  # now typed (Pydantic model)

    def get_file_md5(filepath: str) -> str:
        """Calculates file hash to uniquely identify videos and prevent duplicate
indexes."""
        hasher = hashlib.md5()
        with open(filepath, "rb") as f:
            # Read in chunk sizes of 64kb
            for chunk in iter(lambda: f.read(65536), b""):
                hasher.update(chunk)
        return hasher.hexdigest()

    # Legacy thin wrapper for any remaining direct calls during transition.
    # New code should import load_app_config / AppConfig from backend.config directly.
    def load_config(config_path: str = "config.json") -> Dict[str, Any]:
        cfg = load_app_config(config_path)
        return cfg.model_dump(mode="json")

    # --- API Models ---
    class ProcessRequest(BaseModel):
        video_path: str
        player_pool: Optional[List[str]] = None
        max_frames: Optional[int] = None
        segmenter: Optional[str] = None

    class QueryRequest(BaseModel):
        video_id: str
        server: Optional[str] = None
        receiver: Optional[str] = None
        duration_min: Optional[float] = None
        event_type: Optional[str] = None

    class CompileRequest(BaseModel):
        video_id: str
        segment_ids: List[int]
        output_filename: str

    # --- API Endpoints ---
    @app.get("/api/config")
    def get_config():
        return global_config

```

```
90      @app.post("/api/process")
91      def process_video(req: ProcessRequest):
92          if not os.path.exists(req.video_path):
93              raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
94
95          video_id = get_file_md5(req.video_path)
96          was_reingest = db_instance.get_video(video_id) is not None
97
98          # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
99          print(f"[API] Running validations for {req.video_path}...")
100         val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
101         failures = [r for r in val_results if not r.passed]
102
103         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
104         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
105         seg_name = req.segmenter or default_seg
106         segmenter_actual = None
107         segmentation_ran = False
108         response: Optional[Dict[str, Any]] = None
109         try:
110             if failures:
111                 db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
val_results])
112                 response = {
113                     "video_id": video_id,
114                     "status": "failed",
115                     "message": "Video failed validation checks.",
116                     "results": [r.dict() for r in val_results],
117                 }
118                 return response
119
120             # 2. Run segmenter & indexer
121             print(f"[API] Video passed checks. Segmenting and indexing...")
122             db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])
123
124             segmenter_cls = segmenter_registry.get(seg_name)
125             if not segmenter_cls:
126                 available = list(segmenter_registry.list_available().keys())
127                 raise HTTPException(
128                     status_code=400,
129                     detail=f"Segmenter '{seg_name}' not found. Available segmenters:
{available}"
130                 )
131
132             print(f"[API] Using segmenter: {seg_name}")
133             segmenter_actual = seg_name
134             segmenter = segmenter_cls(db_instance, global_config)
135             segments = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
136             segmentation_ran = True
137
138             response = {
139                 "video_id": video_id,
140                 "status": "success",
141                 "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
142                 "results": [r.dict() for r in val_results],
143                 "play_segments": segments,
144             }
145             return response
```

```
        finally:
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response.
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths

    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")

        # Retrieve matching play segments from db
        all_segments = db_instance.query_play_segments(req.video_id, {})
        matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

        if not matched_segments:
            raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")

        # Extract start/end time pairs
        time_pairs = [(s["start_time"], s["end_time"]) for s in matched_segments]

        # Run compiler
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        compiler = VideoCompiler(output_dir)
        try:
            out_path = compiler.compile_highlights(video["filepath"], time_pairs,
req.output_filename)
            return {"status": "success", "filepath": out_path}
        except Exception as e:
            raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}")

    # --- CLI Debug Utility & Orchestration ---
    def run_cli_diagnose_clip(video_path: str, timestamp: float):
        """CLI debugging utility to examine segment properties around a timestamp."""
        print("=" * 60)
        print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
        print("=" * 60)

        cfg = load_config()  # legacy wrapper still works, returns dict for now

        # 1. Validation check diagnostics
        print("[DIAGNOSTIC] Running validation framework...")
        results = registry.run_all(video_path, cfg)
```

```
206         for r in results:
207             status_symbol = "[PASS]" if r.passed else "[FAIL]"
208             print(f"  {status_symbol} {r.validator_name}: {r.message}")
209             if r.details:
210                 print(f"         Details: {r.details}")
211 
212         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
213         print("-" * 60)
214         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
215         import cv2
216         cap = cv2.VideoCapture(video_path)
217         if not cap.isOpened():
218             print("[FAIL] OpenCV could not open video.")
219             return
220 
221         fps = cap.get(cv2.CAP_PROP_FPS)
222         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
223 
224         start_frame = max(0, int((timestamp - 3.0) * fps))
225         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
226 
227         # Measure frame-by-frame changes in sample region
228         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
229         prev_gray = None
230         motions = []
231 
232         for f in range(start_frame, end_frame):
233             ret, frame = cap.read()
234             if not ret:
235                 break
236             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
237             gray = cv2.GaussianBlur(gray, (21, 21), 0)
238 
239             if prev_gray is not None:
240                 diff = cv2.absdiff(prev_gray, gray)
241                 _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
242                 motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
243                 motions.append(motion_ratio)
244             prev_gray = gray
245 
246         cap.release()
247 
248         if motions:
249             avg_motion = sum(motions) / len(motions)
250             # Support both legacy dict (from wrapper) and future direct model
251             if hasattr(cfg, "indexing"):
252                 threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
253             else:
254                 threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)  # type:
ignore[attr-defined]
255             print(f"  Sample Frame Count: {len(motions)}")
256             print(f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
257             if avg_motion > threshold:
258                 print("  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
259             else:
260                 print("  Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
261         else:
262             print("  [ERROR] No visual frames were extracted in the sample range.")
263 
264         print("=" * 60)
265 
```

```
    def main():
        parser = argparse.ArgumentParser(description="Sports Highlight Indexer Orchestrator")
        parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
        parser.add_argument("--output-dir", default="output", help="Directory where processed clips/highlights are saved")
        parser.add_argument("--setup", action="store_true", help="Launch dependency checker and setup utility")
        parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a video clip to run detailed diagnostics on")
        parser.add_argument("--server", action="store_true", help="Run the FastAPI local API server")
        parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI server on")
        parser.add_argument("--max-frames", type=int, help="Limit processing to the first N sub-sampled frames for debugging")

        available_segmenters = list(segmenter_registry.list_available().keys())
        parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally detection segmenter. Available: {available_segmenters}")
        parser.add_argument("--trim-start", type=float, help="Start time in seconds for the debug trim segment feature")
        parser.add_argument("--trim-end", type=float, help="End time in seconds for the debug trim segment feature")
        parser.add_argument("--trim-output", help="Output filename for the debug trimmed segment (saves in output-dir unless absolute path)")

        args = parser.parse_args()

        if args.setup:
            # Invoke setup diagnostics
            import subprocess
            print("[INFO] Invoking setup.py...")
            subprocess.run([sys.executable, "setup.py"])
            return

        if args.trim_start is not None and args.trim_end is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to extract a segment.")
                return

            # Determine output path
            out_filename = args.trim_output or "trimmed_segment.mp4"
            if os.path.isabs(out_filename):
                out_path = out_filename
                out_dir = os.path.dirname(out_path)
                out_name = os.path.basename(out_path)
            else:
                out_dir = args.output_dir
                out_path = os.path.join(out_dir, out_filename)
                out_name = out_filename

            os.makedirs(out_dir, exist_ok=True)
            print(f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s from {args.video_path}...")

            compiler = VideoCompiler(out_dir)
            try:
                res_path = compiler.compile_highlights(args.video_path, [(args.trim_start, args.trim_end)], out_name)
                print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
            except Exception as e:
                print(f"[ERROR] Failed to compile trimmed segment: {e}")
            return

```

```
        if args.debug_clip is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to debug a specific clip.")
                return
            run_cli_diagnose_clip(args.video_path, args.debug_clip)
            return

        # Initialize Global Config and DB
        global global_config, db_instance
        global_config = load_app_config()  # returns typed AppConfig

        # output_dir is a runtime concern (not stored in config.json).
        # We attach it to the config object for downstream compatibility during transition.
        # Downstream code that does dict access will still work via model_dump() if needed,
        # but new code should read cfg.output.output_dir after we extend the model.
        if not hasattr(global_config, "_runtime_overrides"):
            global_config._runtime_overrides = {}
        global_config._runtime_overrides["output_dir"] = args.output_dir

        # For code that still does global_config.get("output", {}) we provide a thin compat
layer
        # by using the model + overrides when necessary. For now we compute db_path
directly.
        os.makedirs(args.output_dir, exist_ok=True)

        db_name = getattr(getattr(global_config, "output", None), "db_name",
"sports_indexer.db")
        db_path = os.path.join(args.output_dir, db_name)
        db_instance = Database(db_path)

        # CLI processing mode (no web UI needed)
        if args.video_path and not args.server:
            print(f"[CLI] Processing video file: {args.video_path}")
            if not os.path.exists(args.video_path):
                print(f"[FAIL] Video file not found: {args.video_path}")
                return

            video_id = get_file_md5(args.video_path)
            was_reingest = db_instance.get_video(video_id) is not None

            # Validate (with-context variant surfaces ffprobe_meta for the report)
            print("[CLI] Validating...")
            val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
            failures = [r for r in val_results if not r.passed]

            # Save video status
            status = "failed" if failures else "processed"
            db_instance.add_video(video_id, args.video_path, status, [r.dict() for r in
val_results])

            seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
            segmenter_actual = None
            segmentation_ran = False
            try:
                print("\n" + "="*40)
                print("VALIDATION SUMMARY")
                print("="*40)
                for r in val_results:
                    status_symbol = "[PASS]" if r.passed else "[FAIL]"
                    print(f"{status_symbol} {r.validator_name}: {r.message}")
                print("="*40 + "\n")

                if failures:
```

```
377                     print("[FAIL] Video failed checks. Indexing halted.")
378                     return
379
380                 # Index
381                 segmenter_cls = segmenter_registry.get(seg_name)
382                 if not segmenter_cls:
383                     print(f"[FAIL] Segmenter '{seg_name}' not found.")
384                     return
385
386                 print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
387                 segmenter_actual = seg_name
388                 segmenter = segmenter_cls(db_instance, global_config)
389                 segments = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
390                 segmentation_ran = True
391                 print(f"[SUCCESS] Process complete. Indexed {len(segments)} play segments
inside database: {db_path}")
392             finally:
393                 # Always emit the per-video observability report. Never raises.
394                 paths = emit_indexer_report(
395                     db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
396                     val_results=val_results, val_context=val_context,
397                     segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
398                     was_reingest=was_reingest, segmentation_ran=segmentation_ran,
399                 )
400                 if paths:
401                     print(f"[REPORT] Technical report : {paths.get('technical_md')}")
402                     print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
403                     print(f"[REPORT] Machine JSON     : {paths.get('json')}")
404             return
405
406         # Start Local Server Mode
407         if args.server or not args.video_path:
408             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
409             uvicorn.run(app, host="127.0.0.1", port=args.port)
410
411     if __name__ == "__main__":
412         main()
413
```

4. user (2026-06-07T09:10:34.726Z)

```
1       """Configuration package.
2
3       Public API:
4           from backend.config import load_config, AppConfig, Config
5
6       This package is the canonical place for all configuration access.
7       """
8
9       from .loader import load_config, save_default_config
10      from .models import AppConfig, Config, IndexingConfig, ValidationConfig  # re-export
common ones
11
12      __all__ = [
13          "load_config",
14          "save_default_config",
15          "AppConfig",
16          "Config",
17          "IndexingConfig",
18          "ValidationConfig",
19      ]
20
```

5. user (2026-06-07T09:10:35.706Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, Literal
12
13      from pydantic import BaseModel, Field, field_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33      class TrackNetConfig(BaseModel):
34          wsl_distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
40          stage_in_wsl: bool = True
41          wsl_stage_dir: str = "~/clips"
42          timeout_sec: int = 0
43          velocity_threshold: float = 0.02
44
45
46      class IndexingConfig(BaseModel):
47          default_segmenter: str = "yolo_hybrid"
48          use_gpu: bool = True
49          motion_threshold: float = 0.08
50          min_segment_duration: float = 3.0
51          dead_time_merge_gap: float = 2.0
52          chunk_duration: float = 90.0
53          chunk_overlap: float = 10.0
54          detector_model: str = "fasterrcnn_resnet50_fpn"
55          detector_score_threshold: float = 0.5
56          yolo_velocity_threshold: float = 0.15
57          yolo_frame_skip: int = 3
58          yolo_tracker: str = "bytetrack.yaml"
59          yolo_parallel_chunks: bool = False
60          yolo_chunk_workers: int = 2
61          min_action_window_duration: float = 2.0
62          log_yolo_candidates: bool = True
```

```
63          store_yolo_candidates: bool = True
64          ai_handoff_padding: float = 2.0
65          ai_max_workers: int = 5
66
67          tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
68
69          @field_validator("default_segmenter")
70          @classmethod
71          def segmenter_must_be_registered(cls, v: str) -> str:
72              # Registry check happens at runtime in main/segmenter selection.
73              # Here we just allow any string for forward compatibility.
74              return v
75
76
77      class OutputConfig(BaseModel):
78          default_highlights_name: str = "highlights.mp4"
79          db_name: str = "sports_indexer.db"
80
81
82      class StitchingConfig(BaseModel):
83          begin_padding: float = 0.5
84          end_padding: float = 0.5
85          use_cache: bool = False
86          min_shot_count: int = 1
87
88
89      class AppConfig(BaseModel):
90          """Root configuration object.
91
92          All runtime configuration should be accessed via an instance of this class
93          (or the loader) rather than raw dict lookups.
94          """
95
96          sport: Sport = "badminton"
97          ai_provider: str = "gemini"
98          ai_model: str = "gemini-2.5-flash"
99
100         validation: ValidationConfig = Field(default_factory=ValidationConfig)
101         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
102         output: OutputConfig = Field(default_factory=OutputConfig)
103         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
104
105         # Allow extra keys during transition so existing config.json files do not break.
106         model_config = {
107             "extra": "allow",
108             "validate_assignment": True,
109         }
110
111         # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
112         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
113         # attribute access / the typed config. A nested *section* is returned as a plain
dict
114         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
115         # section model instead breaks it (a model has no .get), which bit every validator
on
116         # real runs. Scalar leaves are returned as-is.
117         def get(self, key: str, default: Any = None) -> Any:
118             _MISSING = object()
119             val: Any = _MISSING
120             if key in type(self).model_fields or hasattr(self, key):
121                 val = getattr(self, key, _MISSING)
122             if val is _MISSING:
123                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
```

```
124                 # discovered dynamically from the model fields, so a future top-level
section
125                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
126                 for fname in type(self).model_fields:
127                     section = getattr(self, fname, None)
128                     if isinstance(section, BaseModel) and hasattr(section, key):
129                         val = getattr(section, key)
130                         break
131             if val is _MISSING:
132                 return default
133             if isinstance(val, BaseModel):
134                 return val.model_dump(mode="json")
135             return val
136 
137         def __getitem__(self, key: str) -> Any:
138             val = self.get(key)
139             if val is None and key not in type(self).model_fields:
140                 # Re-raise like a real dict for unknown top-level
141                 raise KeyError(key)
142             return val
143 
144 
145     # Convenience type alias for callers that want to annotate
146     Config = AppConfig
```

6. user (2026-06-07T09:10:36.163Z)

```
1       """Configuration loader and saver.
2 
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5 
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
8 
9       from __future__ import annotations
10 
11      import json
12      from pathlib import Path
13      from typing import Any
14 
15      from .models import AppConfig
16 
17      DEFAULT_CONFIG_PATH = Path("config.json")
18 
19 
20      def _get_builtin_defaults() -> dict[str, Any]:
21          """Return a plain dict of the current built-in defaults.
22 
23          The AppConfig model is the single source of truth for defaults.
24          """
25          return AppConfig().model_dump(mode="json")
26 
27 
28      def load_config(path: str | Path | None = None) -> AppConfig:
29          """Load configuration with the following precedence:
30 
31          1. Built-in defaults defined in the Pydantic models.
32          2. Values from the JSON file (if present and readable).
33          3. (Future) environment / CLI overrides.
34 
35          Missing file or unreadable file ? returns a fully defaulted AppConfig.
36          Unknown keys in the file are tolerated (extra="allow" on the model).
37          """
```

```
        cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH

        file_data: dict[str, Any] = {}
        if cfg_path.exists():
            try:
                with open(cfg_path, "r", encoding="utf-8") as f:
                    file_data = json.load(f)
            except Exception as exc:
                # Non-fatal: continue with defaults + whatever we could parse.
                # In production we might log here.
                print(f"[CONFIG WARNING] Failed to read {cfg_path}: {exc}. Using defaults +
partial data.")

        # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
        merged = {**_get_builtin_defaults(), **file_data}

        return AppConfig.model_validate(merged)


    def save_default_config(path: str | Path | None = None) -> Path:
        """Write a fresh default config.json.

        Used by setup.py diagnostics and the --setup CLI path.
        Overwrites if the file exists.
        """
        cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
        cfg = AppConfig()
        cfg_path.parent.mkdir(parents=True, exist_ok=True)
        cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
        return cfg_path


    # Backwards-compatible alias used by some older call sites during transition.
    def get_default_config() -> dict[str, Any]:
        """Return defaults as a plain dict (for transitional use only)."""
        return _get_builtin_defaults()

```

7. user (2026-06-07T09:10:36.632Z)

```
    """CLI to evaluate detected rallies against hand-annotated ground truth.

    Scores predictions already stored in the SQLite DB (run the pipeline first) ? no
    API calls, fully deterministic. See docs/EVALUATION.md for the workflow.

    Examples
    --------
    Create an empty annotation file to fill in:
        python run_eval.py --scaffold --annotations rallies.csv

    Score a video's detected rallies (identify it by file or by stored video_id):
        python run_eval.py --video "C:/path/match.mp4" --annotations rallies.csv
        python run_eval.py --video-id <md5> --annotations rallies.csv --stage both --iou 0.5

    Show which rallies were missed / spurious, and dump a JSON report:
        python run_eval.py --video-id <md5> --annotations rallies.csv --show-failures --json
report.json
    """

    import os
    import sys
    import json
    import hashlib
    import argparse
    import logging
```

```
25
26      from backend.infrastructure.database import Database
27      from backend.eval.annotations import load_annotations, write_scaffold
28      from backend.eval.harness import evaluate, format_report_text, report_to_dict, STAGES
29
30
31      def get_file_md5(filepath: str) -> str:
32          """MD5 of a file, chunked ? matches how the pipeline derives video_id."""
33          hasher = hashlib.md5()
34          with open(filepath, "rb") as f:
35              for chunk in iter(lambda: f.read(8192), b""):
36                  hasher.update(chunk)
37          return hasher.hexdigest()
38
39
40      def _default_db_path() -> str:
41          """Resolve the DB path from config.json (output.db_name) under output/."""
42          db_name = "sports_indexer.db"
43          if os.path.exists("config.json"):
44              try:
45                  with open("config.json") as f:
46                      db_name = json.load(f).get("output", {}).get("db_name", db_name)
47              except (json.JSONDecodeError, OSError):
48                  pass
49          return os.path.join("output", db_name)
50
51
52      _EPILOG = """\
53      Prerequisite:
54        The harness scores predictions read from the SQLite DB ? it does NOT run the
55        pipeline. So you must first PROCESS the video (e.g. via run_process_and_stitch.py)
56        so its detections land in output/sports_indexer.db. Then point this tool at the
57        same video file (--video) or its stored MD5 (--video-id). If you get
58        "database not found" or zero predictions, the video hasn't been processed yet.
59
60      See docs/EVALUATION.md for the annotation format and how to read the output.
61      """
62
63
64      def build_parser() -> argparse.ArgumentParser:
65          p = argparse.ArgumentParser(
66              description="Evaluate detected rallies vs ground-truth annotations.",
67              epilog=_EPILOG,
68              formatter_class=argparse.RawDescriptionHelpFormatter,
69          )
70          p.add_argument("--annotations", help="Path to the ground-truth rally CSV (start,end
per row).")
71          p.add_argument("--video", help="Path to the video file (its MD5 becomes the
video_id).")
72          p.add_argument("--video-id", help="Stored video_id (MD5) to evaluate, instead of
--video.")
73          p.add_argument("--db", default=None, help="SQLite DB path (default: from
config.json).")
74          p.add_argument("--stage", choices=("candidates", "final", "both"), default="both",
75                         help="Which prediction stage(s) to score (default: both).")
76          p.add_argument("--detector", default=None,
77                         help="Only score candidates from this detector (e.g. yolo_hybrid,
tracknet_hybrid).")
78          p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
79          p.add_argument("--pr-curve", action="store_true", help="Include an IoU sweep
(0.1..0.9) in the report.")
80          p.add_argument("--show-failures", action="store_true",
81                         help="List missed (FN) and spurious (FP) rallies with timestamps.")
82          p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report to
```

```
this path.")
        p.add_argument("--scaffold", action="store_true",
                       help="Write an empty annotation CSV to --annotations and exit.")
        return p


    def main(argv=None) -> int:
        logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
        args = build_parser().parse_args(argv)

        # Scaffold mode: just write the template and exit.
        if args.scaffold:
            if not args.annotations:
                print("ERROR: --scaffold requires --annotations <path>", file=sys.stderr)
                return 2
            write_scaffold(args.annotations)
            print(f"Wrote empty annotation file: {args.annotations}")
            return 0

        if not args.annotations:
            print("ERROR: --annotations is required (or use --scaffold).", file=sys.stderr)
            return 2

        # Resolve the video_id.
        if args.video_id:
            video_id = args.video_id
        elif args.video:
            if not os.path.exists(args.video):
                print(f"ERROR: video not found: {args.video}", file=sys.stderr)
                return 2
            print("Computing video MD5...")
            video_id = get_file_md5(args.video)
            print(f"video_id = {video_id}")
        else:
            print("ERROR: provide --video <path> or --video-id <md5>.", file=sys.stderr)
            return 2

        db_path = args.db or _default_db_path()
        if not os.path.exists(db_path):
            print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
            return 2

        gt = load_annotations(args.annotations)
        if not gt:
            print(f"WARNING: no ground-truth rallies in {args.annotations}.",
file=sys.stderr)

        stages = list(STAGES) if args.stage == "both" else [args.stage]
        db = Database(db_path)
        report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
                          detector=args.detector, with_pr_curve=args.pr_curve)

        print(format_report_text(report, show_failures=args.show_failures))

        if args.json_out:
            with open(args.json_out, "w") as f:
                json.dump(report_to_dict(report), f, indent=2)
            print(f"\nJSON report written to {args.json_out}")

        return 0


    if __name__ == "__main__":
        raise SystemExit(main())
```

8. user (2026-06-07T09:10:37.123Z)

```
"""Phase 3 driver: batch process + evaluate a collector export against ground truth.

Walks the `manifest.json` produced by sports-data-collector's `curate export`,
processes each video through a segmenter (so detections land in the indexer DB),
scores each against its ground-truth rally CSV, and prints per-video plus
corpus-aggregate precision/recall/F1, mean IoU, and boundary error.

Examples
--------
Free CPU baseline on a single video (validate the loop):
    python run_phase3_eval.py --manifest ../sports-data-collector/data/eval_export/manifest.json \
        --segmenter motion --only shuttleset_1

Full corpus with the two-stage detector (needs GEMINI_API_KEY; costs API calls):
    python run_phase3_eval.py --manifest .../manifest.json --segmenter yolo_hybrid \
        --json output/phase3_report.json

Re-score only (videos already processed), no re-segmentation:
    python run_phase3_eval.py --manifest .../manifest.json --score-only
"""

import os
import sys
import json
import time
import hashlib
import argparse
import logging

from dotenv import load_dotenv
load_dotenv()  # make GEMINI_API_KEY (etc.) available to the AI-backed segmenters

from backend.infrastructure.database import Database
# Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
from backend.pipeline.segmenters import segmenter_registry
from backend.eval.annotations import load_annotations
from backend.eval.harness import evaluate, format_report_text, report_to_dict
from backend.eval import batch

logger = logging.getLogger(__name__)


def get_file_md5(filepath: str) -> str:
    hasher = hashlib.md5()
    with open(filepath, "rb") as f:
        for chunk in iter(lambda: f.read(65536), b""):
            hasher.update(chunk)
    return hasher.hexdigest()


def load_config(path: str = "config.json") -> dict:
    if os.path.exists(path):
        with open(path) as f:
            return json.load(f)
    return {}


def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(
        description="Batch process + evaluate a collector export manifest (Phase 3).",
```

```
61              formatter_class=argparse.RawDescriptionHelpFormatter,
62          )
63          p.add_argument("--manifest", required=True, help="Path to the collector's
manifest.json.")
64          p.add_argument("--videos-root", default=None,
65                         help="Root for resolving manifest relative local_paths "
66                              "(default: the collector repo root, two levels above the
manifest).")
67          p.add_argument("--segmenter", default="motion",
68                         help="Segmenter to run (default: motion ? free/CPU). "
69                              "yolo_hybrid/gemini need GEMINI_API_KEY and cost API calls.")
70          p.add_argument("--stage", choices=("candidates", "final", "both", "auto"),
default="auto",
71                         help="Which prediction stage(s) to score (default: auto by
segmenter).")
72          p.add_argument("--detector", default=None, help="Restrict candidate scoring to this
detector.")
73          p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
74          p.add_argument("--only", action="append", default=None,
75                         help="Only this video_id (repeatable).")
76          p.add_argument("--limit", type=int, default=None, help="Process at most N videos.")
77          p.add_argument("--max-frames", type=int, default=None,
78                         help="Cap segmenter frames (fast validation).")
79          p.add_argument("--score-only", action="store_true",
80                         help="Skip segmentation; only score detections already in the DB.")
81          p.add_argument("--reprocess", action="store_true",
82                         help="Re-run segmentation even if the video already has segments.")
83          p.add_argument("--db", default=None, help="Indexer DB path (default:
output/<db_name>).")
84          p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report
here.")
85          p.add_argument("--show-failures", action="store_true", help="List missed/spurious
rallies per video.")
86          return p
87
88
89      def main(argv=None) -> int:
90          logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
91          args = build_parser().parse_args(argv)
92
93          manifest_path = os.path.abspath(args.manifest)
94          if not os.path.exists(manifest_path):
95              print(f"ERROR: manifest not found: {manifest_path}", file=sys.stderr)
96              return 2
97          manifest_dir = os.path.dirname(manifest_path)
98          videos_root = args.videos_root or os.path.dirname(os.path.dirname(manifest_dir))
99
100         with open(manifest_path) as f:
101             manifest = json.load(f)
102         entries = manifest.get("eval_sets", [])
103
104         cfg = load_config()
105         db_name = cfg.get("output", {}).get("db_name", "sports_indexer.db")
106         db_path = args.db or os.path.join("output", db_name)
107         os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
108         db = Database(db_path)
109
110         stages = batch.default_stages_for(args.segmenter, args.stage)
111
112         # Resolve the segmenter class up front (unless purely scoring).
113         segmenter = None
114         if not args.score_only:
115             seg_cls = segmenter_registry.get(args.segmenter)
116             if not seg_cls:
```

```
117                 print(f"ERROR: segmenter '{args.segmenter}' not found. "
118                       f"Available: {list(segmenter_registry.list_available().keys())}",
file=sys.stderr)
119                 return 2
120             segmenter = seg_cls(db, cfg)
121 
122         # Filter entries.
123         targets = []
124         for e in entries:
125             if args.only and e["video_id"] not in args.only:
126                 continue
127             targets.append(e)
128         if args.limit:
129             targets = targets[: args.limit]
130 
131         per_video = []
132         skipped = []
133         print(f"Phase 3: {len(targets)} target(s) | segmenter={args.segmenter} |
stages={stages} | db={db_path}\n")
134 
135         for e in targets:
136             vid = e["video_id"]
137             video_path = batch.resolve_video_path(e.get("local_path"), videos_root)
138             csv_path = os.path.join(manifest_dir, e["csv"])
139 
140             if not video_path or not os.path.exists(video_path):
141                 skipped.append((vid, "no video file"))
142                 print(f"[skip] {vid}: no video file ({e.get('local_path')})")
143                 continue
144             if not os.path.exists(csv_path):
145                 skipped.append((vid, "no GT csv"))
146                 print(f"[skip] {vid}: GT csv missing ({csv_path})")
147                 continue
148 
149             video_id = get_file_md5(video_path)
150 
151             # Segment unless score-only or already present (and not --reprocess).
152             if not args.score_only:
153                 already = db.query_play_segments(video_id, {}) or
db.query_candidate_segments(video_id)
154                 if already and not args.reprocess:
155                     print(f"[have] {vid}: detections already in DB (use --reprocess to
redo)")
156                 else:
157                     print(f"[proc] {vid}: segmenting with {args.segmenter}...")
158                     db.add_video(video_id, video_path, "processed", [])
159                     t0 = time.time()
160                     segs = segmenter.process_video(video_id, video_path,
max_frames=args.max_frames)
161                     print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")
162 
163             gt = load_annotations(csv_path)
164             report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
detector=args.detector)
165             print(format_report_text(report, show_failures=args.show_failures))
166             print()
167             d = report_to_dict(report)
168             d["collector_video_id"] = vid
169             per_video.append(d)
170 
171         agg = batch.aggregate(per_video, stages)
172         print(batch.format_aggregate(agg))
173         if skipped:
174             print(f"\nSkipped {len(skipped)}: " + ", ".join(f"{v} ({why})" for v, why in
skipped))
```

```
175
176         if args.json_out:
177             with open(args.json_out, "w") as f:
178                 json.dump({"segmenter": args.segmenter, "iou": args.iou, "stages": stages,
179                            "aggregate": agg, "per_video": per_video,
180                            "skipped": skipped}, f, indent=2)
181             print(f"\nJSON report written to {args.json_out}")
182
183         return 0
184
185
186     if __name__ == "__main__":
187         raise SystemExit(main())
188
```

9. user (2026-06-07T09:10:37.571Z)

```
1       """CLI: regression-test the production tool against golden-set ground truth.
2
3       Obtains ground truth from an annotation provider (a tier), scores the video's
4       stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
5       detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.
6
7       Examples
8       --------
9       Snapshot the current numbers as the baseline (first run / after a real improvement):
10          python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
--update-baseline
11
12      Check for a regression (exit 1 if precision/recall dropped beyond tolerance):
13          python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
14      """
15      import os
16      import sys
17      import json
18      import argparse
19      import logging
20
21      from backend.infrastructure.database import Database
22      from backend.annotations import annotation_registry
23      from backend.eval import regression
24
25
26      def _default_db_path() -> str:
27          db_name = "sports_indexer.db"
28          if os.path.exists("config.json"):
29              try:
30                  with open("config.json") as f:
31                      db_name = json.load(f).get("output", {}).get("db_name", db_name)
32              except (json.JSONDecodeError, OSError):
33                  pass
34          return os.path.join("output", db_name)
35
36
37      def build_parser() -> argparse.ArgumentParser:
38          p = argparse.ArgumentParser(description="Regression-test the tool vs golden-set
ground truth.")
39          p.add_argument("--video-id", required=True, help="Stored video_id (MD5) whose
predictions to score.")
40          p.add_argument("--tier", default="file",
41                         help=f"Annotation provider tier. Available:
{annotation_registry.list_available()}")
42          p.add_argument("--annotations", help="video_ref for the provider (e.g. CSV path for
the 'file' tier).")
43          p.add_argument("--stage", choices=("candidates", "final"), default="final")
```

```
44          p.add_argument("--detector", default=None, help="Restrict candidate scoring to one
detector.")
45          p.add_argument("--iou", type=float, default=0.5)
46          p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
47          p.add_argument("--db", default=None)
48          p.add_argument("--baseline", default=regression.DEFAULT_BASELINE_PATH,
help="Baseline JSON path.")
49          p.add_argument("--update-baseline", action="store_true",
50                         help="Snapshot the current numbers as the new baseline instead of
checking.")
51          p.add_argument("--json", dest="json_out", default=None)
52          return p
53
54
55      def main(argv=None) -> int:
56          logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
57          args = build_parser().parse_args(argv)
58
59          db_path = args.db or _default_db_path()
60          if not os.path.exists(db_path):
61              print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
62              return 2
63          db = Database(db_path)
64
65          result = regression.regress_with_provider(
66              db, video_id=args.video_id, video_ref=args.annotations, tier=args.tier,
67              stage=args.stage, iou_threshold=args.iou, detector=args.detector,
68              tolerance=args.tolerance, baseline_path=args.baseline,
69          )
70
71          print(regression.format_result(result))
72
73          if args.update_baseline:
74              regression.save_baseline(result.video_id, result.stage, result.precision,
75                                       result.recall, result.f1, path=args.baseline)
76              print(f"Baseline updated in {args.baseline}.")
77
78          if args.json_out:
79              with open(args.json_out, "w") as f:
80                  json.dump(result.as_dict(), f, indent=2)
81              print(f"JSON written to {args.json_out}")
82
83          # Non-zero exit on regression so CI can gate on it.
84          return 1 if result.regressed else 0
85
86
87      if __name__ == "__main__":
88          raise SystemExit(main())
89
```

10. user (2026-06-07T09:10:38.513Z)

```
1       import os
2       import sys
3       import json
4       import hashlib
5       import logging
6       from dotenv import load_dotenv
7
8       load_dotenv()
9
10      # Configure root logger ? all backend modules use logging.getLogger(__name__)
11      logging.basicConfig(
12          level=logging.INFO,
```

```
        format="%(asctime)s [%(name)s] %(levelname)s: %(message)s",
        datefmt="%H:%M:%S",
    )

    from backend.infrastructure.database import Database
    from backend.pipeline.validators.base import registry
    from backend.pipeline.segmenters import segmenter_registry
    from backend.pipeline.stitcher.compiler import VideoCompiler

    def get_file_md5(filepath: str) -> str:
        total_size = os.path.getsize(filepath)
        hasher = hashlib.md5()
        processed = 0
        with open(filepath, "rb") as f:
            # Read in 1MB chunks for faster IO on huge files
            for chunk in iter(lambda: f.read(1024 * 1024), b""):
                hasher.update(chunk)
                processed += len(chunk)
                if processed % (1024 * 1024 * 50) == 0 or processed == total_size:
                    print(f"\r[MD5] Processed {processed/(1024**3):.2f} GB /
{total_size/(1024**3):.2f} GB ({(processed/total_size)*100:.1f}%)", end="", flush=True)
        print()
        return hasher.hexdigest()

    def main():
        video_path = r"C:\Users\avidu\Videos\Testing\KushagraYashAviFirstVideo.mp4"
        output_dir = r"C:\Users\avidu\Videos\Testing"
        output_filename = "Final_2_OutputOfKushagraYashAviFirstVideo.mp4"

        print(f"Loading config...")
        if os.path.exists("config.json"):
            with open("config.json", "r") as f:
                config = json.load(f)
        else:
            config = {}

        print(f"Verifying files exist...")
        if not os.path.exists(video_path):
            print(f"Error: Video file not found at {video_path}")
            return

        os.makedirs("output", exist_ok=True)
        db_path = os.path.join("output", "sports_indexer.db")
        db = Database(db_path)

        # Calculate MD5 of the video to uniquely identify it
        import time
        print("Calculating video MD5 (this might take a few seconds on large files)...",
flush=True)
        t0 = time.time()
        video_id = get_file_md5(video_path)
        print(f"Calculated MD5: {video_id} (took {time.time()-t0:.1f}s)", flush=True)

        stitching_config = config.get("stitching", {})
        use_cache = stitching_config.get("use_cache", True)

        segments = []
        cached_video = db.get_video(video_id)
        cache_hit = False

        if cached_video and cached_video.get("status") == "processed":
            segments = db.query_play_segments(video_id, {})
            if len(segments) > 0:
                print(f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
with {len(segments)} parsed play segments.", flush=True)
```

```
                choice = input("Would you like to (1) Run stitching on the cached segments, or (2) Reprocess the video with Gemini from scratch? [1/2]: ").strip()
                if choice == '1':
                    print("[CACHE] Using cached play segments. Skipping segmenter API call.", flush=True)
                    cache_hit = True
                else:
                    print("[CACHE] Ignoring cache...", flush=True)
            else:
                print(f"[CACHE] Video exists in DB but contains 0 play segments. Treating as cache miss.", flush=True)

        if not cache_hit:
            print("[CACHE] Initializing clean indexing...", flush=True)
            db.clear_video_data(video_id)

            print("Bypassing OpenCV validations for performance on massive files...", flush=True)
            db.add_video(video_id, video_path, "processed", [])

            print("Running segmenter & indexing...", flush=True)
            seg_name = config.get("indexing", {}).get("default_segmenter", "yolo_hybrid")
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(f"[FAIL] Segmenter '{seg_name}' not found.")
                return
            print(f"Using segmenter: {seg_name}", flush=True)
            segmenter = segmenter_cls(db, config)
            segments = segmenter.process_video(video_id, video_path)
            print(f"Successfully indexed {len(segments)} play segments.", flush=True)

        if not segments:
            print("No play segments detected. Cannot compile highlights.")
            return


        end_padding = stitching_config.get("end_padding", 1.0)
        begin_padding = stitching_config.get("begin_padding", 0.5)
        min_shot_count = stitching_config.get("min_shot_count", 0)

        if cache_hit:
            print("\n--- Stitching Customizations ---")
            try:
                bpad_input = input(f"Enter begin_padding in seconds (default {begin_padding}): ").strip()
                if bpad_input:
                    begin_padding = float(bpad_input)

                pad_input = input(f"Enter end_padding in seconds (default {end_padding}): ").strip()
                if pad_input:
                    end_padding = float(pad_input)

                shot_input = input(f"Enter min_shot_count to filter by (default {min_shot_count}): ").strip()
                if shot_input:
                    min_shot_count = int(shot_input)
            except ValueError:
                print("[WARNING] Invalid input detected, falling back to defaults from config.json.")

        print(f"\nCompiling highlights (with begin_padding={begin_padding}s, end_padding={end_padding}s, min_shot_count={min_shot_count})...", flush=True)

        # Filter segments where shot_count >= min_shot_count
```

```
131         time_pairs = []
132         skipped_count = 0
133         for s in segments:
134             meta = s.get("metadata", {})
135             if isinstance(meta, str):
136                 try:
137                     meta = json.loads(meta)
138                 except Exception:
139                     meta = {}
140             shot_count = meta.get("shot_count", 0)
141             if shot_count >= min_shot_count:
142                 time_pairs.append((s["start_time"], s["end_time"]))
143             else:
144                 skipped_count += 1
145 
146         if skipped_count > 0:
147             print(f"[FILTER] Excluded {skipped_count} play segments with fewer than
{min_shot_count} shots.", flush=True)
148         print(f"[FILTER] Retaining {len(time_pairs)} play segments for stitching.",
flush=True)
149 
150         if not time_pairs:
151             print("No play segments passed the filtering criteria. Cannot compile
highlights.")
152             return
153 
154         compiler = VideoCompiler(output_dir, end_padding=end_padding,
begin_padding=begin_padding)
155         try:
156             out_path = compiler.compile_highlights(video_path, time_pairs, output_filename)
157             print(f"Stitched highlight video saved to: {out_path}", flush=True)
158         except Exception as e:
159             print(f"Error during stitching: {e}", flush=True)
160 
161     if __name__ == "__main__":
162         main()
163 
```

11. user (2026-06-07T09:10:38.996Z)

```
1       import sys
2       import os
3       import subprocess
4       import shutil
5       import json
6 
7       def check_command(cmd):
8           """Check if a system command is available."""
9           return shutil.which(cmd) is not None
10 
11      def run_pip_install():
12          """Install required Python packages."""
13          print("Installing Python packages from requirements.txt...")
14 
15          # Check if NVIDIA GPU is present
16          has_gpu = check_command("nvidia-smi")
17 
18          pip_cmd = [sys.executable, "-m", "pip", "install", "--user", "-r",
"requirements.txt"]
19 
20          if has_gpu:
21              print("[INFO] NVIDIA GPU detected. Configuring PyTorch to download CUDA-enabled
wheels.")
22              pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cu124"])
23          else:
```

```
            print("[INFO] No NVIDIA GPU detected. Defaulting to PyTorch CPU wheels.")
            pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cpu"])

        try:
            subprocess.check_call(pip_cmd)
            print("[SUCCESS] Python packages installed.")
            return True
        except subprocess.CalledProcessError as e:
            print(f"[ERROR] Failed to install Python packages: {e}")
            return False

    def init_default_config():
        """Initialize a default config.json if it doesn't exist.

        Now delegates to the canonical backend.config loader so that defaults
        stay in sync with the Pydantic models (single source of truth).
        """
        try:
            from backend.config import save_default_config
        except Exception as e:
            print(f"[ERROR] Could not import backend.config loader: {e}")
            print("[INFO] Falling back to no-op for config initialization.")
            return

        config_path = "config.json"
        if os.path.exists(config_path):
            print(f"[INFO] {config_path} already exists. Skipping initialization.")
            return

        try:
            saved_path = save_default_config(config_path)
            print(f"[SUCCESS] config.json initialized with default settings at
{saved_path}.")
        except Exception as e:
            print(f"[ERROR] Failed to create config.json via loader: {e}")

    def run_diagnostics():
        """Perform system-wide checks for FFmpeg and PyTorch/CUDA."""
        print("=" * 50)
        print("SPORTS HIGHLIGHT INDEXER SETUP DIAGNOSTICS")
        print("=" * 50)

        # 1. Check Python Version
        print(f"Python Version:
{sys.version_info.major}.{sys.version_info.minor}.{sys.version_info.micro}")
        if sys.version_info.major < 3 or (sys.version_info.major == 3 and
sys.version_info.minor < 8):
            print("[WARNING] Python 3.8+ is highly recommended!")
        else:
            print("[PASS] Python version check passed.")

        print("-" * 50)

        # 2. Check FFmpeg / FFprobe
        ffmpeg_ok = check_command("ffmpeg")
        ffprobe_ok = check_command("ffprobe")

        if ffmpeg_ok:
            print("[PASS] FFmpeg is installed and available in PATH.")
        else:
            print("[FAIL] FFmpeg is NOT found in your PATH.")

        if ffprobe_ok:
            print("[PASS] FFprobe is installed and available in PATH.")
        else:
```

```
            print("[FAIL] FFprobe is NOT found in your PATH.")

        if not (ffmpeg_ok and ffprobe_ok):
            print("\n" + "!" * 50)
            print("CRITICAL: FFmpeg and FFprobe are REQUIRED to run this application.")
            print("How to install FFmpeg on Windows:")
            print("  Option 1: Open PowerShell as Administrator and run:")
            print("            winget install Gyan.FFmpeg")
            print("  Option 2: Download the build from https://ffmpeg.org/download.html")
            print("            and add its 'bin' directory to your System PATH environment variable.")
            print("!" * 50 + "\n")
        else:
            print("[SUCCESS] All essential media command line tools are ready.")

        print("-" * 50)

        # 3. Check ML / GPU (Optional but good)
        try:
            import torch
            cuda_ok = torch.cuda.is_available()
            if cuda_ok:
                print(f"[PASS] PyTorch loaded successfully. CUDA (GPU) is AVAILABLE: {torch.cuda.get_device_name(0)}")
            else:
                print("[INFO] PyTorch loaded successfully, but CUDA (GPU) is NOT available. Falling back to CPU.")
        except ImportError:
            print("[INFO] PyTorch is not installed. Relevance checks will run using lightweight visual heuristics.")

        print("=" * 50)
        return ffmpeg_ok and ffprobe_ok

    if __name__ == "__main__":
        init_default_config()
        media_ok = run_diagnostics()
        run_pip_install()
        print("\n[INFO] Setup complete. If diagnostics passed, you can now run the app!")

```

12. user (2026-06-07T09:10:39.192Z)

```
      {
          "sport": "badminton",
          "ai_provider": "gemini",
          "ai_model": "gemini-2.5-flash",
          "validation": {
              "min_resolution_width": 1280,
              "min_resolution_height": 720,
              "min_fps": 29.9,
              "min_blur_threshold": 20.0,
              "max_scene_cuts_per_minute": 0.5,
              "relevance": {
                  "court_green_lower": [
                      35,
                      40,
                      40
                  ],
                  "court_green_upper": [
                      85,
                      255,
                      255
                  ],
                  "court_pixels_min_ratio": 0.05
```

```
            }
        },
        "indexing": {
            "default_segmenter": "gemini",
            "use_gpu": true,
            "motion_threshold": 0.08,
            "min_segment_duration": 3.0,
            "dead_time_merge_gap": 2.0,
            "chunk_duration": 90,
            "chunk_overlap": 10,
            "detector_model": "fasterrcnn_resnet50_fpn",
            "detector_score_threshold": 0.5,
            "yolo_velocity_threshold": 0.15,
            "yolo_frame_skip": 3,
            "yolo_tracker": "bytetrack.yaml",
            "yolo_parallel_chunks": false,
            "yolo_chunk_workers": 2,
            "min_action_window_duration": 2.0,
            "log_yolo_candidates": true,
            "store_yolo_candidates": true,
            "ai_handoff_padding": 2.0,
            "ai_max_workers": 5,
            "tracknet": {
                "wsl_distro": "Ubuntu",
                "repo_dir": "~/models/TrackNetV4",
                "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
                "conda_env": "TrackNetV4",
                "weights_path": "",
                "queue_length": 5,
                "stage_in_wsl": true,
                "wsl_stage_dir": "~/clips",
                "timeout_sec": 0,
                "velocity_threshold": 0.02
            }
        },
        "output": {
            "default_highlights_name": "highlights.mp4",
            "db_name": "sports_indexer.db"
        },
        "stitching": {
            "begin_padding": 0.5,
            "end_padding": 0.5,
            "use_cache": false,
            "min_shot_count": 1
        }
    }
```

13. assistant (2026-06-07T09:12:01.679Z)

```
I have all the files. Producing the dense CODE_MAP chunk now.

---
```

Subsystem: Orchestration & config

```
Entry points (CLI/API/scripts), the typed config layer, and the eval/regression/stitch drivers.
All paths repo-relative.
```

Typed config layer (`backend/config/`)

```
- **@backend/config/__init__.py** ? public config API. Re-exports `load_config`,
`save_default_config` (from `.loader`) and `AppConfig`, `Config`, `IndexingConfig`,
`ValidationConfig` (from `.models`). `__all__` lists exactly these 6. ? Canonical import
surface: `from backend.config import load_config, AppConfig`.
```

- **@backend/config/models.py** ? Pydantic v2 config models; single source of truth for defaults.
  - §`@backend/config/models.py::Sport` ? `Literal["badminton","tennis","pickleball","table_tennis","cricket"]`.
  - §`@backend/config/models.py::ValidationRelevanceConfig` ? court HSV gates: `court_green_lower=[35,40,40]`, `court_green_upper=[85,255,255]`, `court_pixels_min_ratio=0.05` (clamped `ge=0.0,le=1.0`).
  - §`@backend/config/models.py::ValidationConfig` ? `min_resolution_width=1280`, `min_resolution_height=720`, `min_fps=29.9`, `min_blur_threshold=20.0`, `max_scene_cuts_per_minute=0.5`, `relevance` (nested).
  - §`@backend/config/models.py::TrackNetConfig` ? WSL2/conda invocation contract: `wsl_distro="Ubuntu"`, `repo_dir="~/models/TrackNetV4"`, `conda_sh`, `conda_env="TrackNetV4"`, `weights_path=""`, `queue_length=5`, `stage_in_wsl=True`, `wsl_stage_dir="~/clips"`, `timeout_sec=0`, `velocity_threshold=0.02`.
  - §`@backend/config/models.py::IndexingConfig` ? segmenter/detector knobs. ? **default_segmenter="yolo_hybrid"** here, but the on-disk `config.json` overrides it to `"gemini"` (see config.json gotcha). Other fields: `use_gpu`, `motion_threshold=0.08`, `min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`, `chunk_overlap=10.0`, `detector_model="fasterrcnn_resnet50_fpn"`, `detector_score_threshold=0.5`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`, `yolo_tracker="bytetrack.yaml"`, `yolo_parallel_chunks=False`, `yolo_chunk_workers=2`, `min_action_window_duration=2.0`, `log_yolo_candidates=True`, `store_yolo_candidates=True`, `ai_handoff_padding=2.0`, `ai_max_workers=5`, nested `tracknet`.
  - ? `@backend/config/models.py::IndexingConfig.segmenter_must_be_registered` ? `@field_validator("default_segmenter")`; intentionally a **no-op pass-through** (allows any string for forward-compat; registry check deferred to runtime in main/segmenter selection). ? A typo'd segmenter name will validate fine here and only fail later at `segmenter_registry.get(...)`.
  - §`@backend/config/models.py::OutputConfig` ? `default_highlights_name="highlights.mp4"`, `db_name="sports_indexer.db"`. ? No `output_dir` field ? output dir is a **runtime-only** concern (see main.py `_runtime_overrides`).
  - §`@backend/config/models.py::StitchingConfig` ? `begin_padding=0.5`, `end_padding=0.5`, `use_cache=False`, `min_shot_count=1`.
  - §`@backend/config/models.py::AppConfig` ? root model. Scalars: `sport="badminton"`, `ai_provider="gemini"`, `ai_model="gemini-2.5-flash"`; sections `validation`, `indexing`, `output`, `stitching` (all `default_factory`). `model_config = {"extra":"allow","validate_assignment":True}`.
    - ? `extra="allow"` ? unknown keys in config.json are silently kept (legacy-migration tolerance), so typos in section/key names are NOT rejected.
    - ? `validate_assignment=True` ? `global_config._runtime_overrides = {}` in main.py works only because `extra="allow"` permits the dynamic attr; assignment is still validated.
    - ? `@backend/config/models.py::AppConfig.get` ? legacy dict-style shim. Returns nested **sections as plain dicts** via `model_dump(mode="json")` (NOT the Pydantic model) so chained `config.get("indexing",{}).get(...)` keeps working; scalar leaves returned as-is. Falls back to searching a leaf key inside any nested section (sections discovered dynamically from `model_fields`, so a future top-level section is covered without code edits). ? This dict-vs-model footgun "bit every validator on real runs" per the docstring ? returning the model breaks `.get`.
    - ? `@backend/config/models.py::AppConfig.__getitem__` ? `config[key]`; raises `KeyError` for unknown top-level keys (dict-like), else delegates to `.get`.
  - §`@backend/config/models.py::Config` ? alias `Config = AppConfig` (annotation convenience).

- **@backend/config/loader.py** ? load/save with defaults-merge precedence.
  - `@backend/config/loader.py::DEFAULT_CONFIG_PATH` = `Path("config.json")` (CWD-relative). ? Loader resolves config relative to the **current working directory**, not the repo root.
  - `@backend/config/loader.py::_get_builtin_defaults` ? `AppConfig().model_dump(mode="json")`; models are the single source of truth.
  - `@backend/config/loader.py::load_config` ? precedence: built-in defaults ? file overrides ? (future) env/CLI. ? **Shallow top-level merge only** (`{**defaults, **file_data}`): if config.json supplies a section (e.g. `"indexing"`), the entire file section REPLACES the defaults dict for that section before validation ? partial sub-keys are not deep-merged with model defaults at this layer (Pydantic then re-applies field defaults for any missing leaf). Missing/unreadable file ? fully-defaulted `AppConfig` (parse errors print `[CONFIG WARNING]` and continue, non-fatal).
  - `@backend/config/loader.py::save_default_config` ? writes fresh `config.json`

(`model_dump_json(indent=4)`); **overwrites** if present. Used by setup.py / `--setup`.
  - `@backend/config/loader.py::get_default_config` ? ? transitional alias returning defaults as plain dict; "for transitional use only".

- §**@config.json** ? on-disk config; structurally mirrors `AppConfig`. ? **`indexing.default_segmenter="gemini"`** ? diverges from the model default `"yolo_hybrid"`; the file wins at runtime. Note `chunk_duration: 90` / `chunk_overlap: 10` are JSON ints, coerced to float by Pydantic.

App entry point (`backend/main.py`)

- **@backend/main.py::app** ? `FastAPI(title="Sports Highlight Indexer API")`; CORS allow-all; mounts `/static` from `backend/api/static` (dir auto-created).
- `@backend/main.py::db_instance` / `@backend/main.py::global_config` ? module-level globals (`Optional[Database]` / `Optional[AppConfig]`); initialized in `main()`. ? API handlers assume these are non-None ? calling endpoints without going through `main()` (the only initializer) would NPE.
- `@backend/main.py::get_file_md5` ? 64 KB-chunked MD5 ? `video_id`. ? Three copies of this exist with **different chunk sizes**: main.py 65536, run_eval.py 8192, run_phase3_eval.py 65536, run_process_and_stitch.py 1 MB. All produce the same hash (chunk size doesn't affect MD5), but it is duplicated, not shared.
- `@backend/main.py::load_config` ? ? legacy thin wrapper returning a **dict** (`load_app_config(path).model_dump(mode="json")`); kept for transition. New code should import `load_app_config`/`AppConfig` directly.
- §`@backend/main.py::ProcessRequest` ? `video_path`, `player_pool?`, `max_frames?`, `segmenter?`.
- §`@backend/main.py::QueryRequest` ? `video_id`, `server?`, `receiver?`, `duration_min?`, `event_type?`.
- §`@backend/main.py::CompileRequest` ? `video_id`, `segment_ids: List[int]`, `output_filename`.
- `@backend/main.py::get_config` ? `GET /api/config` ? returns `global_config`.
- `@backend/main.py::process_video` ? `POST /api/process`. Flow: md5 ? `registry.run_all_with_context(path, global_config)` ? (validator registry) ? on failure persist `"failed"` and return early ? else pick segmenter via `req.segmenter or global_config.indexing.default_segmenter` (fallback `"motion"`) ? `segmenter_registry.get(seg_name)` ? (segmenter registry; 400 with available list if missing) ? `segmenter_cls(db_instance, global_config).process_video(...)`. ? `finally:` always calls `emit_indexer_report(...)` (never raises) and grafts `response["reports"]` ? report emission is unconditional even on validation failure / exceptions.
- `@backend/main.py::query_play_segments` ? `POST /api/query`; builds filter dict and calls `db_instance.query_play_segments`.
- `@backend/main.py::compile_highlights` ? `POST /api/compile`. ? Reads output dir via `getattr(getattr(global_config,"output",None),"output_dir","output") or "output"` ? but `OutputConfig` has **no `output_dir` field**, so this always falls back to `"output"` (the `_runtime_overrides["output_dir"]` set in `main()` is NOT consulted here). Builds `VideoCompiler(output_dir)`; wraps stitch in try/except ? 500.
- `@backend/main.py::run_cli_diagnose_clip(video_path, timestamp)` ? `--debug-clip` diagnostics: runs `registry.run_all`, samples ±3 s motion via OpenCV, compares avg motion to threshold. ? Uses legacy `load_config()` (dict path) and has a dual-path read: `if hasattr(cfg,"indexing")` (model) else `cfg.get("indexing",{}).get("motion_threshold",0.08)` (dict) ? defensive because the wrapper currently returns a dict.
- `@backend/main.py::main` ? argparse orchestrator. Flags: `--video-path`, `--output-dir`(default `output`), `--setup`, `--debug-clip`(float), `--server`, `--port`(8000), `--max-frames`, `--segmenter` (? `choices` populated dynamically from `segmenter_registry.list_available()` at parse time), `--trim-start/--trim-end/--trim-output`. Modes (priority order): `--setup` shells out to `setup.py` via subprocess ? trim mode (`VideoCompiler.compile_highlights` single pair) ? `--debug-clip` ? otherwise init `global_config=load_app_config()` + `Database`, then CLI-process mode (video-path & not server) ? else server mode (`uvicorn.run`).
  - ? `@backend/main.py` runtime-override hack: `global_config._runtime_overrides = {"output_dir": args.output_dir}` ? output dir lives only on this dynamic dict, NOT in any config model; downstream `db_path = os.path.join(args.output_dir, output.db_name)`.
  - ? CLI-process path mirrors the API `finally:` report-emission contract (always emits via `emit_indexer_report`, prints `technical_md`/`customer_md`/`json` paths).

Standalone orchestration drivers (repo-root scripts)

- **@run_eval.py** ? CLI: score detected rallies in the DB vs hand-annotated GT (no pipeline run, deterministic). ? Does NOT run segmentation ? predictions must already be in the DB (epilog `_EPILOG` warns about this).
  - `@run_eval.py::get_file_md5` (8 KB chunks), `@run_eval.py::_default_db_path` (? reads `config.json` **directly via json.load**, not `backend.config` ? bypasses the typed loader; ? `output/<db_name>`), `@run_eval.py::_EPILOG`, `@run_eval.py::build_parser`, `@run_eval.py::main`.
  - Flags: `--annotations`, `--video`/`--video-id`, `--db`, `--stage{candidates,final,both}`, `--detector`, `--iou`(0.5), `--pr-curve`, `--show-failures`, `--json`, `--scaffold`. Calls `backend.eval.annotations.{load_annotations,write_scaffold}` and `backend.eval.harness.{evaluate,format_report_text,report_to_dict,STAGES}`. Returns exit code int.
- **@run_phase3_eval.py** ? Phase 3 batch driver: process+evaluate a collector `manifest.json` export, per-video + corpus aggregate.
  - `load_dotenv()` at import (GEMINI_API_KEY for AI segmenters). `@run_phase3_eval.py::get_file_md5` (64 KB), `@run_phase3_eval.py::load_config` (? **own** json.load-based dict loader, not `backend.config`), `@run_phase3_eval.py::build_parser`, `@run_phase3_eval.py::main`.
  - Flags: `--manifest`(required), `--videos-root`(default two levels above manifest), `--segmenter`(default `motion`), `--stage{candidates,final,both,auto}`(default `auto`), `--detector`, `--iou`, `--only`(repeatable), `--limit`, `--max-frames`, `--score-only`, `--reprocess`, `--db`, `--json`, `--show-failures`.
  - §Manifest contract: top-level `eval_sets[]`, each entry `{video_id, local_path, csv}`. ? `segmenter_registry.get(...)` resolution; `segmenter_cls(db, cfg)` where **cfg is a raw dict** (not AppConfig). Uses `backend.eval.batch.{default_stages_for,resolve_video_path,aggregate,format_aggregate}` + harness. ? Skip-rather-than-fail per entry (missing video/CSV ? `skipped`); idempotency via existing `query_play_segments`/`query_candidate_segments` unless `--reprocess`.
- **@run_regression.py** ? CLI: regression-gate the tool vs golden-set GT; **exit 1 on regression** (CI gate).
  - `@run_regression.py::_default_db_path` (? again direct json.load of config.json), `@run_regression.py::build_parser`, `@run_regression.py::main`.
  - Flags: `--video-id`(required), `--tier`(default `file`; ? help lists `annotation_registry.list_available()`), `--annotations`(video_ref for provider), `--stage{candidates,final}`, `--detector`, `--iou`(0.5), `--tolerance`(`regression.DEFAULT_TOLERANCE`), `--db`, `--baseline`(`regression.DEFAULT_BASELINE_PATH`), `--update-baseline`, `--json`. ? `backend.annotations.annotation_registry` (provider-tier plugin point). Calls `backend.eval.regression.{regress_with_provider,format_result,save_baseline,DEFAULT_TOLERANCE,DEFAULT_BASELINE_PATH}`; result fields `.video_id/.stage/.precision/.recall/.f1/.regressed/.as_dict()`. ? `return 1 if result.regressed else 0`.
- **@run_process_and_stitch.py** ? end-to-end one-shot: process a hardcoded video ? segment ? stitch highlights.
  - `@run_process_and_stitch.py::get_file_md5` (1 MB chunks, prints GB progress), `@run_process_and_stitch.py::main`.
  - ? **Hardcoded paths** in `main()`: `video_path=C:\Users\avidu\Videos\Testing\KushagraYashAviFirstVideo.mp4`, `output_dir=...\Testing`, `output_filename="Final_2_OutputOf..."`. Not parameterized ? edit-to-run script.
  - ? Config read via raw `json.load` (dict), NOT `backend.config`. ? `use_cache` default here is `True` (`stitching.get("use_cache",True)`) ? opposite of `StitchingConfig.use_cache=False` and config.json's `false`.
  - ? **Interactive `input()` prompts** (cache reuse 1/2; begin/end padding + min_shot_count overrides) ? blocks on stdin, unsuitable for headless/CI. ? `end_padding` default fallback `1.0` here vs config/model `0.5`; `min_shot_count` fallback `0` here vs config/model `1`.
  - Filters segments by `metadata.shot_count >= min_shot_count` (handles `metadata` as dict OR JSON string). ? `segmenter_registry.get(seg_name)` with `seg_name = config.get("indexing",{}).get("default_segmenter","yolo_hybrid")` (? literal fallback `yolo_hybrid` differs from config.json's `gemini`). `VideoCompiler(output_dir, end_padding=, begin_padding=)`.

Setup / bootstrap

- **@setup.py** ? dependency checker + config initializer; run directly or via `main.py --setup`.

  - `@setup.py::check_command` (`shutil.which`), `@setup.py::run_pip_install` (? GPU-aware: `nvidia-smi` present ? CUDA wheels `cu124`, else CPU wheels; `pip install --user -r requirements.txt`), `@setup.py::init_default_config` (? delegates to `backend.config.save_default_config` ? single-source defaults; **skips if config.json exists**, so it won't clobber), `@setup.py::run_diagnostics` (Python ?3.8, ffmpeg/ffprobe in PATH ? flagged CRITICAL/REQUIRED, optional torch+CUDA probe).
  - `__main__`: `init_default_config()` ? `run_diagnostics()` ? `run_pip_install()`.

Cross-cutting gotchas (orchestration-wide)

- ? **Two config-access idioms coexist.** `backend/main.py` uses the typed `AppConfig` (attribute access + `getattr` fallbacks); all four `run_*.py` scripts read `config.json` via raw `json.load` into a dict and bypass `backend.config` entirely. Default-drift between the model, `config.json`, and per-script literal fallbacks is the recurring footgun (segmenter, paddings, `min_shot_count`, `use_cache`).
- ? **Segmenter default is defined in 5 places with 3 different values:** model `yolo_hybrid`, config.json `gemini`, main.py fallback `motion`, phase3 default `motion`, process_and_stitch fallback `yolo_hybrid`.
- ? **Three runtime registries** are the orchestration extension points: validator `registry` (`backend.pipeline.validators.base`), `segmenter_registry` (`backend.pipeline.segmenters[.base]`, `.get`/`.list_available`), and `annotation_registry` (`backend.annotations`, `.list_available`).
- ? `--segmenter` CLI `choices` in `main.py` are frozen at argparse-build time from the registry ? a segmenter registered later (lazy import) wouldn't appear.

Note on `tests/`

The task listed `tests/` for enumeration, but no `tests/` directory was among the provided paths and none of the read files reference one. No test files were inspected; mapping omitted to avoid fabrication. (If a test suite exists, it should be enumerated in a follow-up pass against the actual directory.)